

Project 1: Python Core Engineering

Project Title: DevOps-Style Automation Engine

Duration: 7 April 2026 – 21 April 2026

Deadline: 21 April 2026

1. Project Overview

In real-world companies, many operational tasks are repetitive. These include organizing files, generating reports, sending emails, and monitoring systems. These tasks are often done manually, which leads to wasted time and human errors.

In this project, you are required to build a **Python-based automation engine** that can handle such tasks automatically using configurations.

This project simulates a **mini DevOps automation system**, where tasks can be scheduled or triggered by events (such as file changes).

2. Problem Statement

Organizations need a system that can:

- Automatically execute tasks at specific times
- React to changes in files or directories
- Perform operations like sending emails, processing files, or generating logs
- Be configurable without modifying code

Your goal is to build a system that solves these problems in a structured and scalable way.

3. Project Objectives

By completing this project, you should be able to:

- Design and build a modular Python application
- Work with real-world automation scenarios
- Understand how background jobs and schedulers work
- Implement clean and maintainable code architecture
- Handle errors and logging in production-style systems

4. What You Need to Build

You are required to build a **command-line based automation engine** that works using configuration files.

The system should allow users to define tasks in a config file and execute them automatically.

5. Functional Requirements

Your system must include the following components:

5.1 Command Line Interface (CLI)

- Users should be able to run commands such as:
 - Start the automation engine
 - Stop the engine
 - View active tasks
- CLI should be user-friendly and well-documented

5.2 Config-Based Workflow System

- Use JSON or YAML for defining workflows
- Example of what a config might include:
 - Task name
 - Task type (email, file processing, etc.)
 - Schedule (e.g., every 5 minutes)
 - File path (if applicable)

The system should read this config and execute tasks accordingly.

5.3 Task Scheduler

- Implement time-based execution:
 - Example: Run a task every 10 minutes
 - Example: Run at a specific time daily
- Scheduler must run continuously in the background

5.4 File Monitoring System

- Monitor a folder for changes such as:

- New file added
 - File modified
- Trigger tasks when changes occur

Example:

- If a new file is added → process it automatically

5.5 Logging System

- Maintain logs for all actions:
 - Task started
 - Task completed
 - Errors
- Logs should be written to a file
- Include timestamps and clear messages

5.6 Email Notification System

- Send emails when:
 - A task is completed
 - A task fails
- Use SMTP for sending emails

6. Technical Requirements

You must follow these constraints:

- Use **pure Python** (avoid heavy frameworks)
- Use:
 - `threading` or `multiprocessing` for parallel execution
- Structure your code into modules:
 - CLI module
 - Scheduler module
 - Task handlers
 - Utilities (logging, email, etc.)

7. Suggested Architecture (Guidance Only)

You may structure your project like this:

- main.py (entry point)
- cli/
- scheduler/
- tasks/
- config/
- utils/

Each task type (email, file processing, etc.) should be a separate module.

8. Deliverables

By the deadline, you must submit:

1. Fully working CLI-based automation tool
2. Sample configuration files
3. Log files showing system activity
4. GitHub repository containing:
 - Clean code
 - Proper commit history
 - README file

9. README Requirements

Your README must include:

- Project description
- How to install and run the project
- Explanation of config file format
- Example commands
- Screenshots or terminal outputs

10. Evaluation Criteria

Your project will be evaluated based on:

10.1 Code Architecture

- Is the code modular and well-structured?

- Is it easy to extend?

10.2 Error Handling

- Does the system handle failures gracefully?
- Are errors logged properly?

10.3 Functionality

- Do all required features work correctly?

10.4 Code Quality

- Clean, readable, and maintainable code

10.5 Extensibility

- Can new task types be added easily?

11. Bonus (For High Performers)

These are not required but will significantly increase your evaluation:

- Plugin system for adding custom tasks
- Retry mechanism for failed tasks
- Parallel execution of multiple tasks
- Web dashboard (optional advanced feature)

12. Important Instructions

- Do not copy code from tutorials
- Focus on understanding and building from scratch
- Ask questions when stuck, but attempt first
- Follow deadlines strictly

This project is intentionally designed to be challenging. Completing it properly will give you a strong foundation in Python and real-world system design.